Nazza Product SOP

Product and Engineering Team
Author: Adedamola Adewale
Last updated: 19th June, 2025

Quick Links

Project links
Nazza 2.0 Go Live Status
Product Team Core Feature Roadmap '25
Nazza Team OKR Q1 '25
Product Requirements Documentation
Changelog
Knowledge base
How to work with the CEO
How to work in a product Team
Product Management Journal: Everything you need to know about Product Management

Table of Content

Purpose

This Standard Operating Procedure (SOP) establishes operational, technical, and team-based procedures for the Nazza platform's maintenance, development, deployment, support, and scaling.

Scope

This document encompasses all product features, user journeys, DevOps activities, data infrastructure, release protocols, growth operations, and collaboration workflows between engineering, product, growth, and design teams.

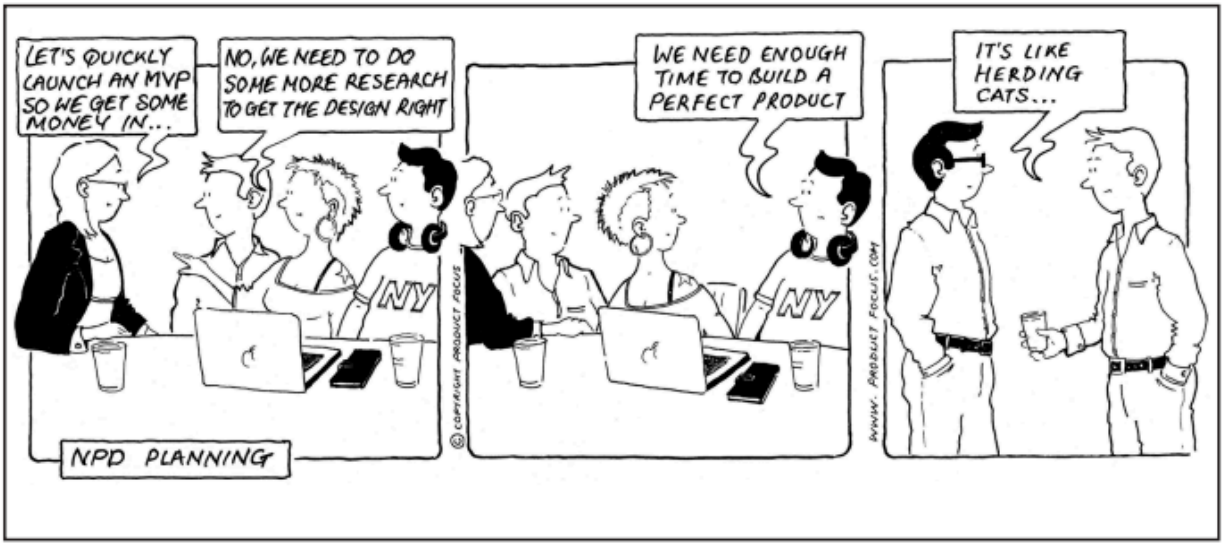
Roles & Responsibilities

Role	Responsibilities
Product Manager (PM)	- Define and own roadmap - Prioritize tasks via backlog - Collect feedback, write PRDs - Run sprints & grooming
Frontend Engineer	- Implement UI features - Integrate with APIs - Fix bugs and performance issues
Backend Engineer	- API development - Payment, authentication, and user logic - Scalability and security
AI Engineer	- Build and train AI models.
QA Engineer	- Test features (manual & automated) - Write test cases and report bugs
Growth Engineer	- Funnel integration - Tracking, A/B testing
Scrum Master	- Run sprint ceremonies - Maintain JIRA hygiene - Ensure blockers are cleared
Customer Support	- Handle user tickets - Send response via Emails, etc

Core Procedures

Feature Development Workflow

1. PM creates a PRD using the Nazza PRD template.
   > Our PRD is here
2. Design team creates Figma prototype, reviewed by dev & PM.
3. Frontend + Backend engineers break down into tasks in JIRA.
   > Watch how we work with Jira
4. Product Manager and Engineering Lead, split into sprints.
5. QA writes test cases from PRD.
6. Development builds, pushes to **Dev**, QA tests.
7. Push to **Staging**, UAT by product.
8. If passed, merge and deploy to **Production**.



How the PM sets his backlogs

Story Creation and Sprint Guidelines
When a 'Story' is created within a two-week sprint, the following guidelines apply:

- **Owner:** A Backend Engineer is the ultimate owner.
- **Deadline:** The overall deadline is 2-4 weeks. If a story exceeds two weeks, it should be divided into phases.
- **Sprint Capacity:** We must not exceed 140 points in a sprint to avoid overworking the team.
- **Documentation:** A Product Requirements Document (PRD) and a design link must be attached.
- **Discovery:** Discovery is conducted within the Engineering Teams.

- **Task Completion:** No Engineer can mark a task as 'done' except QA, Product Manager, and Scrum Master.

Child Issues and Timelines

The following are child issues associated with a 'Story', along with their assigned roles and maximum timelines:

- **Design (Web and Mobile):** Assigned to the Designer in Charge. Timeline: 2 Days Max.
- **Endpoint Development:** Assigned to the Engineer in charge. Timeline: 1 Week Max (including research).
- **Front End UI Implementation (Web and Mobile):** Assigned to the Engineer in charge. Timeline: 3 Days Max.
- **Integrate Endpoint:** Assigned to the Engineer in charge. Timeline: 3 Days.
- **Testing and Review:** Assigned to the QA in charge. Timeline: 1 Day.
- **Go Live:** Last Day of the Sprint. Assign to Product Manager or leave empty.

Ticket Priority and Estimated Points

Tickets are prioritized as follows, with estimated points:

- **Bugs/Support (1 point):** Must be completed within the same day.
- **Story (4 points):** Needs to be achieved within that sprint.
- **Task (2 points):** Can be picked up anytime (usually requests from external stakeholders or operations).

Scrum and Agile

- **Sprint Details:** Sprints last 2-4 weeks. The team develops requirements set by the Product Owner.
- **Sprint Planning:** Held before each sprint to finalize, discuss, and estimate requirements, and agree on deliverables.
- **Daily Stand-up:** A 15-minute meeting run by the Scrum Master to track progress and address blockers. Attendees answer: What did you do? What will you do? What's blocking you? Issues are resolved offline by the Scrum Master.
- **Backlog Refinement:** Often held mid-sprint to discuss and estimate future requirements, aligning priorities and resolving ambiguities for the Product Owner.
- **Sprint Demo:** After the sprint, a demo is held for stakeholders to show deliverables, get Product Owner sign-off, and receive feedback. "Done" means requirements to pass acceptance tests and meet standards. A broader definition might include release notes, user documentation, packaging, and technical debt recording.
- **Sprint retrospective meeting:** This meeting reviews sprint performance, forecast accuracy, and identifies issues. It's an opportunity to implement the Agile mindset through experiments, like workshops to understand problem areas better. Stakeholders outside the core team are not invited.

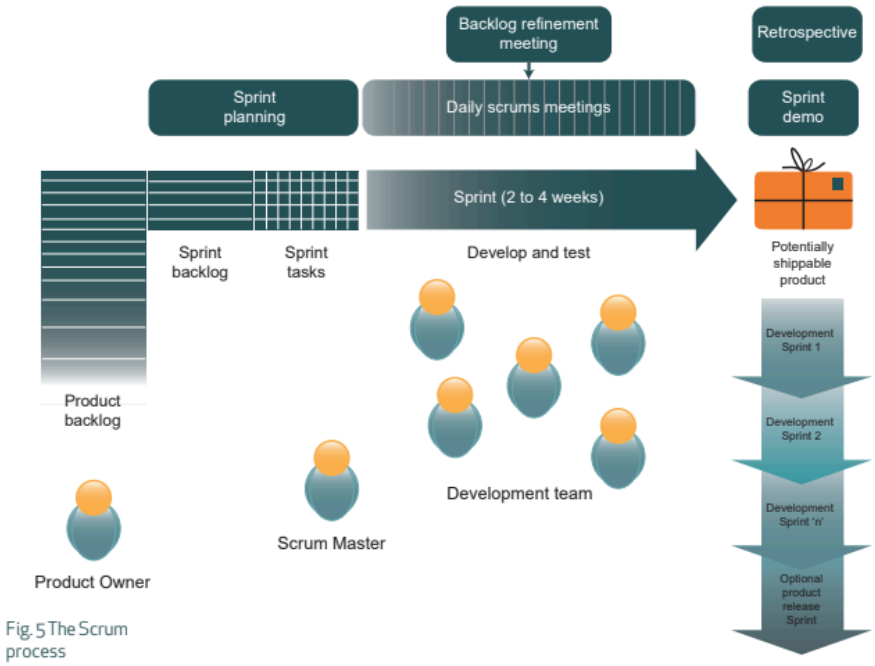


Fig. 5 The Scrum process

Engineering Development

- **PRD Review:** The Product Manager shares the Product Requirements Document (PRD) for the engineering team to review and provide technical feedback.
- **JIRA Task Setup:** The development team breaks down features into individual tickets (Front-end, Back-end, Quality Assurance) and adds effort estimates.
- **Environment Setup:** Developers ensure their local environments precisely mirror the staging environment. Guidelines for `.env` files and configurations are provided during onboarding.
- **Git & Version Control:**
    - Feature branches must adhere to naming conventions (e.g., `feature/xyz`, `bugfix/abc`).
    - Commits should always include clear, descriptive messages (e.g., `feat: added user login modal`).
    - Pull Requests (PRs) require peer review and must pass Continuous Integration (CI) checks before merging.
    - Direct commits to the `main` or production branches are strictly prohibited.
- **Development & Testing:**
    - Code is pushed to the development environment.
    - Unit tests are written for all critical logic.
    - Engineers conduct testing using both mock and real data.
    - The QA team executes test cases derived from the PRD.
- **Deployment Flow:**
    - Once testing is complete and passed, the code is pushed to the staging environment.

    - The Product Manager or Product team conducts User Acceptance Testing (UAT).
    - The final deployment to production occurs via GitHub Actions (CI/CD).
- **Post-Release:**
    - Logs are monitored for early identification of any issues.
    - The QA team confirms the deployed version and tags the release.
    - Retrospective feedback is logged in JIRA.

Mobile App

Code Management & Versioning

- **Branching Strategy:** Maintain distinct Git branches for `main` (production-ready code), `dev` (development and integration), and feature-specific branches.
- **Semantic Versioning:** Implement semantic versioning (e.g., `v1.2.3`) for all application releases.
- **Release Tagging:** Ensure all releases are tagged in Git *before* the build process.

Automated Build Processes

- **CI/CD Tooling:** Utilize Fastlane for comprehensive iOS and Android CI/CD automation.
- **Build Triggering:** Initiate builds via GitHub Actions (or Bitrise, as an alternative).
- **Pre-Deployment Checks:** All builds must successfully pass linting, unit tests, and comprehensive build checks prior to deployment.

Environment Configuration Management

- **Configuration Files:** Leverage `.env` files with React Native Config (or native BuildConfig/Info.plist for platform-specific projects).
- **Environment Segregation:** Maintain separate configurations for development, staging, and production environments.
- **Secret Management:** Securely store sensitive information (secrets) in GitHub Secrets or Firebase Remote Config; *never* hardcode them.

App Distribution Channels

- **Development Builds:** Distribute to developers via TestFlight (iOS) or Firebase App Distribution (Android).
- **Staging Builds:** Provide to the internal QA team through private testing tracks.
- **Production Releases:** Perform manual deployment to the Apple App Store and Google Play Console, ensuring detailed release notes are included.

Crash Reporting & Monitoring

- **Integration:** Integrate robust crash reporting tools such as Sentry or Firebase Crashlytics.
- **Real-time Alerts:** Configure real-time alerts for all critical crashes and application

exceptions.
- **Version Correlation:** Tag releases within the crash reporting system to easily correlate errors with specific code versions.

Beta Testing & User Feedback

- **Closed Beta Programs:** Conduct closed group beta testing, implementing mechanisms for tracking and collecting feedback.
- **In-App Bug Reporting:** Implement an in-app prompt to facilitate bug reporting during User Acceptance Testing (UAT).
- **Release Notes:** Include a comprehensive changelog either directly within the application or on the respective app store listing.

Store Compliance & Maintenance

- **Policy Monitoring:** Continuously monitor and adapt to updates in Apple and Google's app store policies.
- **Expiration Reminders:** Utilize automated reminders for certificate, provisioning profile, and keystore expirations.
- **Documentation Storage:** Maintain all screenshots, app descriptions, and compliance documents in a centralized location (e.g., Notion or Google Drive).

Rollback Strategy

- **Archive Previous Builds:** Maintain an archive of previous APK/IPA files for potential rollbacks.
- **Remote Feature Control:** Implement the ability to remotely disable affected features using Feature Flags or Remote Config in case of issues.

Design

- PM assigns feature with PRD and context.
- Research and create low-fidelity wireframes.
- Develop high-fidelity Figma designs using the design system.
- Design the prototype of the flow for both web and mobile responsiveness
- Conduct internal review with Development and PM.
- Hand off to Development with Figma inspect and design specs.
- Test and Review Staging build for accuracy confirmation.

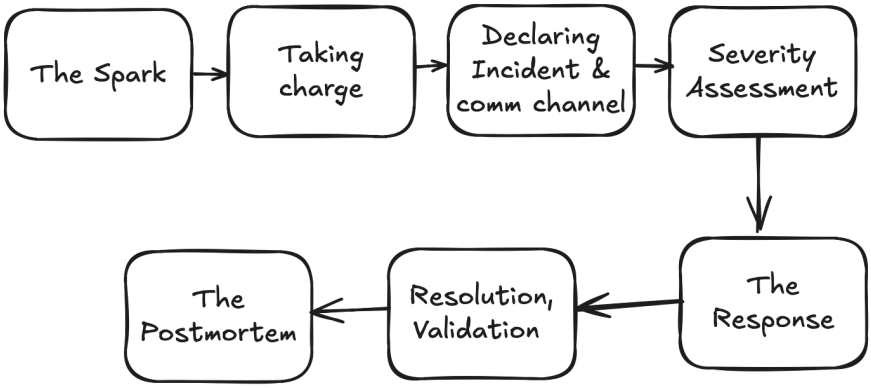
Dev Ops

- **Environment Provisioning**
    - Utilize standardized templates (e.g., Docker, Terraform) for setting up staging and production environments.

    - Strictly enforce `.env` configuration rules for each environment.
    - Monitor all environments using comprehensive metrics dashboards (e.g., Datadog, CloudWatch).
- **CI/CD Pipeline**
    - Automate the build, test, and deployment processes using GitHub Actions (or GitLab CI).
    - Mandate that all Pull Requests (PRs) successfully pass CI checks before merging.
    - Implement auto-deployment for development environments, with manual triggers required for staging and production.
- **Secrets & Access Control**
    - Securely store all credentials in Vault or environment-specific secrets.
    - Ensure monthly rotation of all keys.
    - Adhere to the principle of least-privilege for IAM role assignments and conduct weekly audits of access logs.
- **Monitoring & Alerts**
    - Establish uptime monitors for all critical components including APIs, databases, and frontend.
    - Configure WhatsApp Group alerts for 4xx/5xx errors, deployment failures, and detected downtime.
    - Share daily monitoring reports with the Engineering Lead.
- **Logging & Debugging**
    - Employ centralized logging tools (e.g., LogRocket, ELK) for comprehensive log collection.
    - Tag all logs by their respective release version for easy tracking.
    - Automate log snapshots during deployments to aid in debugging.
- **Backup & Rollback**
    - Perform nightly database backups, retaining them for 30 days.
    - Conduct monthly tests of rollback scripts to ensure their efficacy.
    - Guarantee that all deployments are reversible, leveraging Git tags for easy rollbacks.
- **Security Best Practices**
    - Regularly run dependency scans using tools such as Snyk or Dependabot.
    - Apply patch updates promptly in every sprint.
    - Conduct regular Vulnerability Assessment and Penetration Testing (VAPT) and complete the associated checklist signoff.

QA: Incident Management

1. Issues are reported when tested manually or via monitoring (e.g., Sentry, UptimeRobot) or by users.
2. Logged immediately to Jira, Story Type: **Bugs** by the QA
3. Record tests, screenshots are not efficient, and attach severity.
4. Level 1 triage by engineering on-call (rotation).
5. If urgent, we bring it to **#Product team** on WhatsApp Group, alert CTO/PM.



Change Management

1. All production-affecting changes must have a PRD or documented change log.
   > Here is our changelog so far
2. Notify relevant teams 24 hours before pushing changes.
3. Schedule downtime only during off-peak hours.
4. Emergency changes require CTO/PM approval.

Release Management

- Weekly release cycle (Monday 4PM WAT).
- QA signs off prior to merge.
- Announce new changes in WhatsApp Group **#General Group** and update changelog.
- Announce via changelog notification systems (e.g. email users, General Company-wide Channel)

Task Intake from Other Stakeholders

No product team member (designer, engineer, QA) is allowed to accept or implement tasks directly from any stakeholder without PM approval.

If a stakeholder (Ops, Marketing, Growth, etc.) has a request:

1. **They must submit it to the Product Manager** as a requirement or feature request.
2. The Product Manager will **review, prioritize**, and, if valid, **create a Jira/ClickUp ticket.**
3. Only after it's been approved and documented can it be assigned to a team member.

What is NOT allowed:

- "Quick fixes" or requests over chat that bypass product review.
- Engineers/designers building or tweaking features just because someone asked.
- Implementing anything not tracked in Jira/ClickUp.

"If it's not in the backlog or sprint board, it doesn't exist."

Reporting Guidelines for Engineers, QA, Designers

Every update **must include the task link** (Jira/ClickUp) and **clear context** on what was done.

Example:

1. Fixed: Admin role issue not saving correctly
Jira: https://deexoptions.atlassian.net/browse/BUG-142

2. Started: UI for security center integration
Jira: https://deexoptions.atlassian.net/browse/FEAT-201

QA Reporting: Every QA report must include:

- List of tested tickets with Jira links
- APK/screenshot/test logs as proof of test
- Confirmation that issues are fixed and retested

Key Rules:

- No Jira/ClickUp ticket = It didn't happen.
- Every bug fix, task update, or new feature must have:
  - A ticket
  - Status updated (To Do → In Progress → Done)

Link shared in daily reports

Bug Handling

Severity	Response Time
P0 - System Down, Prod Bugs	30 minutes
P1 - Payment/Revenue Affected	2 hours
P2 - Minor Functional Bugs	24 hours
P3 - Cosmetic/UI	Next release cycle

Product Owners

Why Product Owners are Essential:

- **Clear Accountability:** A single individual is responsible for the product's overall success.
- **Streamlined Communication:** Provides a central point of contact, simplifying interactions and decision-making.
- **Deep Product Expertise:** Fosters comprehensive knowledge of the product, leading to well-informed strategic choices.

Key Responsibilities of a Product Owner:

- **Quality Assurance:** Ensures the product consistently meets quality benchmarks both before and after launch.
- **Product Mastery:** Maintains in-depth familiarity with the product's features, functionality, and user base.
- **Strategic Decision-Making:** Makes informed choices regarding product features, prioritization, and necessary trade-offs.
- **Central Communication Hub:** Acts as the primary liaison for all product-related inquiries and concerns.

Communication Protocols

Team Communication

- Daily standup (10 AM Mon, Wed, Fri).
- Weekly sprint review on Fridays (every 2 weeks, close of sprints).
- The team writes weekly product recap (T5).

Escalations

- System issues: alert via QA WhatsApp Group**.**
- Transaction monitoring and system stats via Telegram Group **#payment.**
- User complaints: auto-logged from Support Channels to WhatsApp group **#technical-support.**
- Code merge conflicts: escalated via GitHub PR comments.
- Payment issues: forward to Paystack/Stripe + alert backend.

Security & Compliance

- All sensitive data encrypted at rest and in transit (AES-256 / SSL).
- OAuth-based login (Google, LinkedIn), + optional 2FA.
- GDPR-compliant storage for EU users.

- Admin access requires logging via IP and device control.
- Database backups run daily; retention: 30 days.
- PCI-DSS compliance for payment handling.

Quality Assurance

- QA tests before merging to staging.
- Regression tests run for every major feature.
- Automated testing: Katalon.
- Load testing for APIs before public releases.
- Bug tracking via Jira: "Bug" label mandatory.

Continuous Improvement

- Biweekly retrospective by product/engineering teams.
- Feature usage tracked on Amplitude, reports sent every Friday.
- Quarterly roadmap review and backlog grooming.
- Postmortems documented in Google docs.
- Quarterly surveys for team and user feedback.

Customer Support Workflow

- Tier 1: Zendesk AI chatbot (24/7).
- Tier 2: Human handoff within 12 hours.
- Tier 3: Escalation to Product/Ops for blockers.
- Ticket types: Billing, Technical, Refund, Feature Request.
- Internal Knowledge Base: Google docs, updated monthly.

Deployment & Versioning

- Web app: Auto-deploy via GitHub → Vercel.
- Mobile app: Deployed via App Store / Play Store.
- Desktop app: Electron build pipeline.
- Version naming convention: vYYYY.MM.DD-feature.

Tools & Resources

Area	Tool
Design	Figma
Task Management	Jira, WhatsApp Group
Source Control	GitHub
Deployment	Vercel (Web), Docker, AWS
Comms	WhatsApp Group, Google Meet
Testing	Postman, Katalon
Monitoring	Sentry
Analytics	Amplitude, Looker Studio
CRM & Support	Tawkto
Documentation	Google Docs. Confluence

Appendix: SOP Ownership by Team

Department	Responsibility Summary
Product	Roadmap, PRDs, Testing Sign Off
Engineering	Feature dev, maintenance, security
QA	Test plans, bug triage, validation
Growth	Funnels, Amplitude tracking, GHL
Support	chatbot flows, ticket resolution
Design	Mockups, UI updates, mobile & web compliance
Ops	WhatsApp Group alerts, admin dashboards, compliance